What survives when a paper leaves the lab: re-testing 22 robot-learning papers on a €1,350 rig, and auditing our own pre-registration

Ravindu Somawansa
Shisso Robotics
ravindu.somawansa@shissorobotics.com*

Abstract

Re-testing a robot-learning paper does not need a lab. For $45.10 of compute on a €1,350 rig, an SO-101 arm with a Jetson Orin NX, we re-tested the central claims of recent papers under pass marks committed before each run, and every published number rebuilds from stored rows. Of the 37 paper claims tested, 12 held on this hardware. The failures fit three proposed classes, each named after the condition the claim did not survive. The clearest is the clock: methods that work on a desktop GPU but miss the real-time budget on the edge board. The other two are zero-shot judges beaten by a trivial signal and latent planners beaten by the commanded joints. A blind second reader still has to confirm the classes. We also audited our own pre-registration by commit ancestry, and the audit shows where our ordering holds and where the repository cannot prove it.

1 The register

A robot-learning paper reports what happened on one setup, and most of the 22 papers here sit far from ours. Of these, 9 were shown in simulation and another 7 on rigs above $20k. Just 2 used a setup under $5k, the bracket our rig falls in (the appendix table gives each paper's rig, including the 3 in between and the 1 that does not say). Ours costs €1,350: an SO-101 arm and a Jetson Orin NX 16 GB, with 2 small parts not yet priced.

The register grew out of our own write-ups, 29 so far, each published with its data. This paper collects them in one table and audits how the table was kept.

Take one entry. An adaptive chunking paper claims that choosing the chunk length from the entropy of sampled chunks follows the task's phases, long in transport and short at the grasp. On our edge GPU a single decision takes 4,751 ms. The budget is 298 ms. So the entry records a fail on this rig, which says nothing against the method on the hardware it was shown on, and beside the verdict it keeps the claim in the paper's words, our pass mark, our measurement and the published rows that rebuild it.

Before a learned model gets credit, the cheapest signal it must beat runs on the same rows: copy-last for a predictor, the commanded joints for a latent, episode length for a judge. A method that loses to that signal has not shown what it claims, however good its own number looks. The threshold itself is committed before the run, and §2 checks how often that held.

We chose the papers for the stack this rig runs (world models, VLA policies, failure monitors, chunking, edge latency) and entered each one before testing it. The next ones are already listed in the register file.

2 Auditing our own pre-registration

To rule out moving the goalposts after the fact, a gate's threshold had to be in version control before its result. Notes and commit dates can be edited by hand. Commit ancestry cannot, short of rewriting history that others have already cloned, so the order of each gate is read from `git merge-base --is-ancestor` alone, between the first commit that contains its bar and the first that contains its result. On the 89 gates logged by 2026-10-31, the audit finds most of them bar-first and 0 result-first (Table 1). Same-commit says nothing about which was written first; it only means bar and result entered the repository together.

Text search alone had found 31 same-commit gates, and batch logging was the reason. A bar written earlier in a protocol, plan, scorer or specification often reached the log in the same commit as its result. Tracing each one back to that file moved 21 to bar-first and 4 to no-bar (their bar cell declares none), and left 6 where they were; the audit table in the repository gives the evidence line for every move. Rows that were already bar-first were not re-audited for a result hidden elsewhere. One open candidate remains, the reward-model progress test of 2026-11-01: same-commit by text search, although its protocol

*Every number with its source file: `github.com/ravediamond/paper-register-reproducibility`.

Table 1: Commit-order class of each gate. The audited column is the one the paper uses; the first pass is what text search alone gave on the same gates.

class	first text pass	audited, 2026-10-31	audited, today
bar-first	36	58	61
same-commit	31	6	7
result-first	—	0	0
no bar	19	23	23
bar, no result yet	—	2	2
unresolved	3	—	—
gates	89	89	93

Table 2: The three gates whose verdict flips somewhere inside its 95 % interval.

gate	n	estimate	95 % CI	bar	point verdict
colour judge (no learning), accuracy	87	94.2 %	89.6–98.9 %	$\geq$ 95 %	fail
oscillation among failed episodes	25	24.0 %	8.0–40.0 %	$\geq$ 10 %	pass
judge threshold, false interventions	42	2.4 %	0.0–7.1 %	$\leq$ 5 %	pass

(`6241570`) is older than the log commit.

Point estimates give clean verdicts, and intervals do not always agree with them. Recomputing 10 early gates from stored rows with one statistic (episode bootstrap, 2,000 resamples, percentile 95 % CI) leaves 3 with the bar inside the interval (Table 2). The colour judge is the one to look at, a fail on the point that its interval cannot settle. For the false-intervention rate the interval only describes, since that threshold was fitted to meet it. The register keeps the pre-registered point verdicts and flags all three as fragile.

All 29 write-ups rebuild from their published rows. Cold reruns are a stricter test. From a fresh clone with no credentials, 6 gates rerun and match, 5 of them exactly. Another 12 stop at the same place, the world-model environment, which is not published yet; publishing it would unblock most of them. The remaining 11 need a larger GPU, a private checkpoint, the arm or a step the write-up leaves out. Citations and paper-side fields were checked against the papers' PDFs and corrected. Of the 38 arXiv ids the register names, those missing from our papers mirror all belong to scheduled tests, and the one tested paper without an arXiv id was not checked against a PDF.

3 Results

Table 3 counts the verdicts, and the appendix lists every entry. The figure we lead with counts only claims taken from papers: 12 of 37 reproduce and 18 fail. The other 8 rows test our own baselines, for instance that the commanded joints beat the world model, and 5 of those hold.

Table 3: Verdicts by paper and by claim row (claim rows include our own baseline comparisons).

verdict	papers	claim rows
reproduces	4	17
fails	13	20
not proven	3	3
not tested	1	4
withdrawn	1	1
total	22	45

4 Three failure classes

The adaptive chunking entry of §1 is the plainest kind of failure: the method may be sound, but on the edge GPU it misses the real-time budget the claim needs (4,751 ms per decision against 298 ms). Nothing about the idea is refuted there; the board is too slow for it. We call this class A, the clock.

A second kind turned up whenever a model judged outcomes without training on this rig. A learned reward model detects failures at AUROC 0.818, where episode length alone reaches 0.938, and judges of success, failure or progress that lose to a trivial signal like this one make up class B. The commanded joints did the same to learned latents. A world model ranks the executed chunk first in 0.60 of anchors, below its pass mark of 0.8 and below the 0.68–0.90 of an action-only baseline, which puts it in class C: a latent used to plan, rank or track progress that loses to the joints or to a copy-last or action-only baseline.

These examples illustrate the classes; they do not count them. A protocol committed before any labelling says the labels count only once a blind second reader, given the same 20 failed rows on the same form, agrees with the first reading at Cohen's $\kappa \geq 0.6$. That reading has not happened yet.

5 Limitations

With a single operator, the author, running the rig and labelling outcomes at rest and not always blind to the arm, observer bias is the main confound. Two things limit it. Where commit order shows it, the pass mark predates the run, so a label could not have moved the bar. A blind two-reader protocol for the failure classes is also committed and ready, though it has not run, and until it does no verdict has a second reader. Ordering is provable for most gates, not all: for 29 of the 89 the repository cannot show which came first, and 21 of the bar-first verdicts rest on matching a bar text in an earlier file. Under the bootstrap 3 verdicts are fragile. Cold reruns cover 6 gates so far, with the unpublished world-model environment as the main blocker. A failure here describes this rig, not the paper in its own setting, and because the papers were chosen for one stack the rate describes this register rather than the field.

6 Related work

Work on the same arm evaluates policies. A standardized real-world benchmark reports a taxonomy of failures and recoveries for VLA policies [13], and a fleet of SO-101 cells evaluates policies at scale [14]. The vla-eval tool runs VLAs across 14 simulation benchmarks [8]. A benchmark of demonstration quality studies one method family [22]. We test something else. Our targets are the claims behind methods (judges, world models, latency), checked against thresholds fixed before the run, with our own pre-registration audited. The closest statement of our thesis [27] scores VLA and world-action models on five dimensions, among them reproduction evidence and deployment affordability. Its reproduction-evidence score counts five audit milestones (configuration mapping, task records, task coverage, training records, seeds), and its authors state that it is not a probability that a result is correct. We give one pass or fail per claim on a €1,350 rig instead.

7 Conclusion

A €1,350 rig and $45.10 of compute were enough to re-test these claims and publish every row behind the verdicts, and 12 of 37 held. Money was never the constraint. Order was: keeping each pass mark ahead of its result, which our own history cannot prove for 29 of 89 gates. If you re-test a paper on a cheap rig, start with its cheapest baseline on the same rows, and commit the threshold before you look. Every number here can be rebuilt from the repository (`github.com/ravediamond/paper-register-reproducibility`).

References

[1] DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning, 2024. arXiv:2411.04983.

[2] Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies, 2025. arXiv:2503.08558.

[3] SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025. arXiv:2506.01844.

[4] Real-Time Execution of Action Chunking Flow Policies, 2025. arXiv:2506.07339.

[5] SAFE: Multitask Failure Detection for Vision-Language-Action Models, 2025. arXiv:2506.09937.

[6] VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model, 2026. arXiv:2602.10098.

[7] Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons, 2026. arXiv:2603.02115.

[8] vla-eval: A Unified Evaluation Harness for Vision-Language-Action Models, 2026. arXiv:2603.13966.

[9] LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels, 2026. arXiv:2603.19312.

[10] Hierarchical Planning with Latent World Models, 2026. arXiv:2604.03208.

[11] Adaptive Action Chunking at Inference-time for Vision-Language-Action Models, 2026. arXiv:2604.04161.

[12] When Does LeJEPA Learn a World Model?, 2026. arXiv:2605.26379.

[13] Benchmarking Vision-Language-Action Models on SO-101: Failure and Recovery Analysis, 2026. arXiv:2606.08881.

[14] ArmnetBench v0.1: Parallel Real-World Evaluation of Manipulation Policies on a Low-Cost Arm Farm, 2026. arXiv:2607.24481.

[15] TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM, 2026. arXiv:2607.27205.

[16] ContactGuard: Pre-Contact Execution Monitoring with Action-Conditioned Latent World Models, 2026. arXiv:2608.13438.

[17] VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies, 2026. arXiv:2609.06508.

[18] D-JEPA: A Decision-Aligned Latent World Model, 2026. arXiv:2609.24749.

[19] H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning, 2026. arXiv:2610.06805.

[20] RoboJEPA: Scaling Robotic Latent World Models, 2026. arXiv:2610.10515.

[21] ReAP: From Visual Failure Recognition to Action-Conditioned Risk Prediction for Runtime Failure Monitoring, 2026. arXiv:2610.16309.

[22] What Matters in Quality-Aware VLA Pretraining: A Benchmark and Empirical Study, 2026. arXiv:2610.18869.

[23] REKI: Registering Knowledge in VLAs, 2026. arXiv:2610.22178.

[24] PACT-VLA: Speculative Action-Chunk Verification with Pre-Commit Observations, 2026. arXiv:2610.28019.

[25] Coupling Execution Horizon and Sampling Depth in Flow-Matching VLA Policies, 2026. arXiv:2610.32102.

[26] Tracking Task Progress Through Errors and Recovery:A Benchmark and Recurrent Distributional Model, 2026. arXiv:2610.39125.

[27] Xin Ma et al. RoboSeed: Multidimensional Evaluation of Vision-Language-Action and World-Action Models, 2026. arXiv:2610.40026.

Appendix: the register

paper	their rig	claim tested	bar	our result	verdict
LeWorldModel [9]	sim	A small JEPA world model learns the arm's dynamics well enough to plan with it.	Dynamics: beat copy-last, every corruption ≥ 1.3 at $K = 30$. Planning: rank the executed chunk first among 8 in ≥ 0.8 of anchors.	dynamics 0.31 vs 0.89 copy-last (reproduces); planning top-1 0.60 vs 0.68–0.90 for an action-only baseline (fail)	fail
DINO-WM [1]	sim	A predictor on frozen DINOv2 patches plans better than one on a learned latent.	G1b top-1 ≥ 0.8 at some target distance.	arm-pose R^2 0.92 (vs 0.57 for W1); G1b top-1 0.55–0.61; no-model baseline 0.68–0.90	fail
RoboJEPA [20]	>$20k	Pretrained RoboJEPA predicts our latents, uses the actions, and runs on the robot's computer.	9-step L1 ≥ 15 % below copy-last (300M) and ≥ 5 % below shuffled actions; 2-view encoder ≤ 200 ms on the Orin.	11.8 % below copy-last (300M; 1B 15.8 % on 30 clips); 9–10 % below shuffled; encoder 526 ms on the Orin NX	fail
H-JEPA [19]	sim	A slow second level makes latent distance track task progress.	Progress monotonicity Spearman $\rho \leq -0.80$; false stalls ≤ 10 %.	level-2 ρ -0.73; false stalls 52 %	fail
D-JEPA [18]	<$5k	A decision-aligned ranking operator on a frozen world model picks better action chunks.	G1b regret ≤ 0.10 and $\leq$ the action-only baseline + 0.05.	regret 0.263 (W1 alone 0.263; action-only -0.007)	not tested
ContactGuard [16]	$5–20k	An event-anchored probe on imagined latents predicts grasp failure better than the current latent.	AUROC ≥ 0.85 and ≥ 0.05 above the current latent.	0.70 imagined vs 0.82 current	fail
FAIL-Detect and SAFE [2] [5]	>$20k	A conformal band on a learned failure score keeps false alarms at α while catching failures.	Successes flagged $\leq 1/20$; failures flagged ≥ 50 %.	22.7 % of successes flagged at $\alpha = 5$ %; 46 % of failures caught	fail
ReAP [21]	$5–20k	A risk head on an action-conditioned latent warns before failures.	$\geq 5/8$ failures caught at $\leq 1/20$ false interventions; action-conditioned iff true $\geq$ shuffled + 0.05.	v1 AUROC 0.93 came from time and length; v2 0.789 vs 0.708 shuffled, 3/8 at 5/42	fail
VLA-Corrector [17]	sim	A verifier on observable history flags a failing rollout early enough to correct it.	$\geq 5/8$ failures at $\leq 1/20$ false interventions.	the paper's verifier family on our rollouts (a GRU on pooled RGB statistics and actions): 2/8 caught at 3/42 false interventions, AUROC 0.59; cosine drift of the W1 latent 0/8, AUROC 0.30	fail
Robometer [7]	>$20k	A 4B reward model judges SO-101 rollouts zero-shot.	Pooled AUROC ≥ 0.85 and within-policy ≥ 0.8 on ≥ 5 of 7 policies.	pooled AUROC 0.818 on 2099 ArmnetBench rollouts; episode length alone 0.938	fail
SmolVLA [3]	<$5k	A 450M VLA succeeds on an SO-101 task and can run on low-cost hardware.	10-trial operator-judged gate (no numeric pass mark registered); chunk inference within the async budget (667 ms).	9/10 on bowls; 974 ms in PyTorch, 238 ms with vla.cpp on the Orin NX	reproduces

Table 4: The register, one row per paper, generated from the register file (internal model and gate names kept). Read the verdict column against the rig each paper was shown on.

paper	their rig	claim tested	bar	our result	verdict
TurboVLA [15]	sim	Our claim that the architecture made the policy ignore its instruction.	Task swap $\geq 5°$ and per-layout $\geq$ 75 % $\rightarrow$ our port was the bug; $< 1°$ with the control $\geq 0.095 \rightarrow$ the data.	upstream's own trainer on our data: task swap 0.020°, per-layout 38.9 %; control, their released LIBERO checkpoint run here: 0.105 ($\approx 5.5°$ of pan)	withdrawn
VLA-JEPA [6]	sim	VLA-JEPA trains cheaply on SO-101 data and runs on the robot.	10-trial operator-judged gate (no numeric pass mark registered); for reranking (H0): oracle / mean error $\leq$ 0.7 and N = 8 $\leq 1.5\times$ the latency.	9/10 for $9.7; 355 ms per chunk on the Orin NX; reranking no-go (0.76, 1.84×)	reproduces
REKI [23]	sim	Policies without memory oscillate between poses at a measurable rate.	$\geq$ 10 % of failed episodes oscillate (their criterion).	20.3 % of all episodes, 24.0 % of failures; REKI trained on my data: 19/20 + 19/20 on the arm, oscillates in 1/40	reproduces
Real-Time Chunking [4]	unstated	Conditioning a new chunk on the executing one removes the jump at chunk boundaries.	Best gradient-free method cuts the median jump $\geq$ 50 % against naive async, drift $\leq$ +10 %, within 667 ms.	prefix overwrite cuts the jump 71–73 % against naive async at +0.4 ms; tuned LeRobot identity mode is close	not proven
HWM (hierarchical latent world models) [10]	>$20k	A high-level world model over learned macro-actions beats the low-level model beyond about 1 s (their Fig. 6 crossover).	Crossover in [1.0, 2.0] s; at 2 s high-level error $\leq$ 0.95 × low-level; learned macro-actions $\geq$ net joint displacement on cosine.	no crossover: W1 0.382 ± 0.005 $\rightarrow$ 0.563 ± 0.007, high level 0.769 ± 0.009 $\rightarrow$ 0.759 ± 0.009 (0.73–2 s); cosine 0.608 ± 0.007 vs 0.551 ± 0.006	not proven
LeJEPA identifiability [12]	sim	A SIGReg JEPA trained on goal-directed data recovers the true latent state only partially and unevenly.	W1's mean R^2 from latent to joints < 0.80; anisotropy (max − min per-joint R^2) ≥ 0.20; the higher-R^2 model has the higher G1b top-1.	W1 mean R^2 0.565, anisotropy 0.639 (shoulder lift 0.91, wrist flex 0.27); frozen DINOv2-S 0.867 / 0.301	reproduces
SURVEYOR	sim	A threshold on the imagined-vs-reached latent distance accepts good plans and rejects bad ones ($\tau = 0.20$).	τ below the 10th percentile of non-equivalent pairs and ≤ 0.40; as a monitor $\geq$ 5/8 failures at $\leq$ 2/42; $\geq$ 0.05 AUROC over a shuffled drafter.	$\tau = 1.249$ on W1 latents of 69 real rollouts; as a monitor 3/8 at 6/42 (S = 30), AUROC 0.756 vs shuffled 0.560	fail
Adaptive Action Chunking (AAC) [11]	>$20k	Chunk length chosen from the entropy of N sampled chunks follows the task's phases (long in transport, short at the grasp), lowers the boundary jump, and costs little with batched sampling.	median h* in transport $\geq$ grasp window + 2; jump with AAC + overwrite $\leq$ fixed + overwrite at d = 8; an N = 20 decision $\leq$ 298 ms on the Orin.	h* 2 in transport and 2 at the grasp (86 % of decisions at the floor); jump 0.973 vs 0.940; N = 20 decision 4,751 ms (sequential) on the Orin	fail
PACT-VLA [24]	$5–20k	Picking among N sampled chunks by their flow residual under the pre-commit observation fits what happens next better than a random or stale pick, with no added control-step delay.	PACT beats a random pick and the stale-observation residual at d = 8 (CIs < 0); its delay on the Orin $\leq$ the naive delay; at that delay it still beats a single sample.	offline $-0.27°$ vs random and vs the stale residual (both CIs < 0); on the Orin 1,200 ms per cycle, 37 control ticks vs 8	fail
Horizon × depth (Euler steps vs execution horizon) [25]	>$20k	With long open-loop commitment, cutting a flow-matching VLA's Euler steps from 10 to 2 contracts the action cloud around an unchanged centroid, saves most of the inference time and avoids edge-GPU thermal throttling.	median R(2)/R(10) ≤ 0.70 with the centroid within 0.05 relative and 3°; N = 2 latency $\leq 0.85 \times$ N = 10 on the Orin, N = 10 within 5 % of 238 ms.	R(2)/R(10) 0.78 (CI 0.73–0.81), centroid 0.018 and 2.35°; 196.1 vs 237.5 ms per chunk (0.83, vision 151 ms); no throttling at N = 10 on the Orin, at MAXN_SUPER or 25 W	not proven
REMORA [26]	>$20k	Reading a progress distribution out at the supported peak nearest the previous estimate steadies the estimate and lets it fall during real setbacks, with no worse detection of the stages.	Against per-frame argmax: premature not higher and missed $\leq$ +2 points (P1); flips halved, CI upper ≤ 0.7 (P2); a decrease inside $\geq$ 50 % of the reversal windows with $\leq$ 20 % false regressions (P3).	premature 0.22 vs 0.25, missed equal (P1 pass); flips 0.70 × argmax, CI 0.48–0.94 (P2 fail); setback recall 2/7, false 0.52 (P3 fail); MAE 0.184 vs median 0.218	fail

Table 4, continued.